Gossip Protocols

spreading news through a cluster like a rumour

Watch the 4-min video

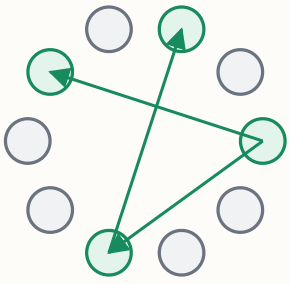
In one line: in a gossip protocol every node periodically swaps state with a few random peers, so information reaches all n nodes in about log n rounds with no central server; it's used for membership, failure detection and metadata, and gives eventual, not instant, agreement.

1 No central server

A cluster has a thousand nodes. A new node joins, or one dies. How does everyone find out? A central server could tell everyone, but it becomes a bottleneck and a single point of failure. Instead, nodes can spread the news the way a rumour spreads: each one tells a few others, who tell a few others. That's a **gossip** (or epidemic) protocol.

2 How it works

Every interval (say, one second), each node picks one to three peers at random and exchanges a summary of what it knows, each entry tagged with a version number such as a heartbeat counter. Each side keeps the newer version of every entry. Because the number of informed nodes roughly multiplies each round, news reaches the whole cluster in **O(log n)** rounds, while each node only does constant work per round.

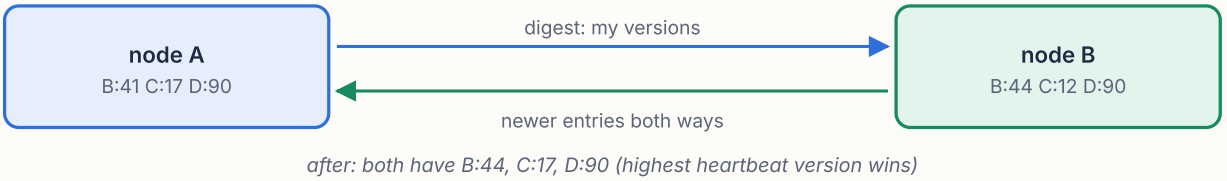


round 2: 4 of 10 know

each round, every node that knows tells a few random peers:

- 10 nodes → about 3 rounds
- 100 nodes → about 5 rounds
- 1,000 nodes → about 8 rounds
- 10,000 nodes → about 11 rounds

Figure 1. Exponential spread: a thousand nodes learn something in about ten rounds.



after: both have B:44, C:17, D:90 (highest heartbeat version wins)

Figure 2. One exchange: the nodes compare versions and both end up with the newest of each.

3 What clusters gossip about

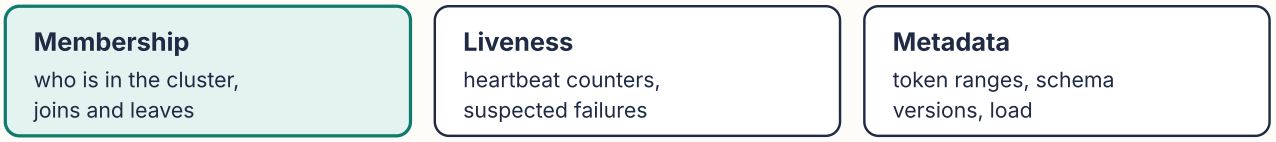


Figure 3. Typical gossiped state. Big payloads still go through normal replication.

WORKED EXAMPLE: CASSANDRA CLUSTER MEMBERSHIP

Every second, each Cassandra node gossips with up to three other nodes, exchanging its view of every node's state: heartbeat generation and version, status (joining, normal, leaving), token ranges and schema version. When node 12 joins, a seed node hears it first; within a few seconds every node knows node 12 and which data it owns. If node 7's heartbeat version stops increasing in everyone's gossip, the phi accrual detector (lesson 4) marks it down. **Seed nodes** are just well-known starting points for new nodes, not a central authority.

4 Trade-offs

- **Robust and scalable:** no single point of failure; lost messages are repaired in later rounds.
- **Eventually consistent:** for a few seconds, nodes may disagree about membership.
- **Bandwidth grows with state size:** send digests first and only the differences.
- Variants: **push**, **pull**, **push-pull**, and **SWIM** (used by Consul and Serf), which adds indirect probing for accurate failure detection.

COMMON MISTAKES

- Using gossip for data that needs strong consistency, like a leader decision.
- Gossiping full state every round in a large cluster.
- Too few seed nodes, or all in one zone.

5 Key terms

Gossip / epidemic protocol: random peer-to-peer state exchange

Digest: summary of versions sent before full data

SWIM: gossip membership with indirect probes

Fan-out: peers contacted per round

Seed node: well-known contact point for joining

Convergence: all nodes reach the same view

6 Interview questions

Q1. How does a gossip protocol spread information? BASIC

Each node periodically exchanges state with a few random peers and keeps the newer version of each entry; the news spreads exponentially, reaching everyone in about log n rounds.

Q2. Why use gossip instead of a central coordinator? BASIC

No bottleneck or single point of failure, constant work per node, and natural tolerance to lost messages and failed nodes.

Q3. How would 5,000 nodes learn about a new node within seconds? SCENARIO

Gossip every second with a fan-out of 2 to 3; about 13 rounds of doubling reach 5,000, so roughly 10 to 15 seconds in the worst case, faster with push-pull.

Q4. Would you use gossip to elect a leader for writes? SCENARIO

No. Gossip gives eventual agreement, so two nodes could both think they lead for a while. Use consensus (Raft, etcd) for that.

Q5. How does gossip handle conflicting information? TRICKY

Every entry has a version, such as a heartbeat counter or generation number, and the higher version wins on merge.

Q6. What limits gossip at very large scale? TRICKY

Bandwidth and CPU grow with the size of the state, not just the node count. Digests, deltas and hierarchy keep it manageable.

Next: SD-H-015 · Distributed Transactions Overview backendroadmap.com